

UNIVERSIDAD CENTRAL DEL ECUADOR

FACULTAD DE CIENCIAS



Maestría en Inteligencia Artificial y Ciencia de Datos

JUAN ANDRÉS MORALES DOMÍNGUEZ

18/09/2026

¿Qué es una API? Define API (interfaz de programación de aplicaciones) y explica con un ejemplo cotidiano (una app del clima, un pago en línea, un chatbot) quién pide y quién responde.

1. Definición

API significa Application Programming Interface (interfaz de programación de aplicaciones). Es un conjunto de reglas que permite que una pieza de software funcione junto con otras (Mozilla Contributors, s. f.-a). Funciona de la siguiente manera:

- Request: La aplicación envía una petición, en la cual pide un dato o una acción específica, como puede ser la consulta del clima o el ingreso a redes sociales mediante Google, la página solicita a Google acceso.
- El intermediario: La API recibe la solicitud, interpreta las reglas y se conecta al programa que tiene la información.
- Response: El programa o sistema procesa la petición y la API entrega la información solicitada.

2. Ejemplo Cotidiano

Cuando compras algo por internet, la tienda casi nunca procesa tu tarjeta por su cuenta. Se conecta mediante una API a una plataforma de pagos, como Kushki, Datafast o PayPhone (Monkey Plus, 2026). Esa plataforma se comunica con el banco emisor, que revisa tus fondos y tus límites y responde "aprobado" o "rechazado". Aquí la tienda es quien pide y la plataforma de pagos es quien responde.

¿Qué es JSON? Explica el formato, muestra un ejemplo de JSON con al menos un objeto, una lista y un número, y di a qué estructuras de Python corresponde (diccionario, lista). Indica qué hace json.loads() y json.dumps().

1. Definición

JSON (JavaScript Object Notation) es un formato de intercambio de datos parecido a la sintaxis de JavaScript, aunque muchos lenguajes de programación lo soportan (Mozilla Contributors, s. f.-c). Por eso casi cualquier lenguaje puede leerlo y escribirlo (Python Software Foundation, s. f.).

```
{
  "nombre": "Ana Pérez",
  "edad": 28,
  "activa": true,
  "ciudad": "Quito",
  "direccion": {
    "calle": "Av. Amazonas",
    "numero": 123
  },
  "telefonos": ["0991234567", "022345678"],
  "saldo": 150.75,
  "pareja": null
}
```

- Un objeto: el JSON completo y también dirección, que es un objeto anidado.
- Una lista: telefonos.
- Números: edad (entero) y saldo (decimal).

json.loads() convierte (deserializa) una cadena de texto con formato JSON en un objeto de Python (como un diccionario). (Python Software Foundation, s. f.).

json.dumps() convierte (serializa) un objeto de Python (como un diccionario o una lista) en una cadena de texto con formato JSON. (Python Software Foundation, s. f.).

¿Qué es CRUD? Explica las cuatro operaciones (Create, Read, Update, Delete) con un ejemplo sobre una tabla de personas encuestadas.

CRUD es la sigla de las cuatro operaciones básicas que se pueden hacer con los datos almacenados en una base de datos o en un sistema (Create, Read, Update y Delete) (Mozilla Contributors, s. f.-b).

En la tabla de encuestados, con el ejemplo de Carlos:

Operación	Qué se hace	Ejemplo
Create	Agregar un registro	El encuestador ingresa sus datos y aparece una fila nueva
Read	Consultar sin modificar	El equipo revisa cuántos respondieron "Sí"
Update	Corregir un registro existente	Carlos corrige su respuesta y su edad
Delete	Eliminar un registro	Carlos pide que borren sus datos y la fila desaparece

- Cada persona es una fila, y el id permite identificarla para modificarla o borrarla.
- Read es la única operación que no cambia la tabla.
- Update cambia los valores, pero el registro sigue siendo el mismo.
- Delete debe hacerse con cuidado, porque muchas veces no se puede deshacer.

Métodos HTTP: GET, POST, PUT y DELETE. Explica qué hace cada uno, relaciónalo con CRUD (qué método corresponde a cada operación) y menciona la diferencia entre PUT y PATCH.

HTTP es el protocolo con el que se comunican los clientes y los servidores en la web. Cada petición incluye un método que indica qué acción se quiere hacer sobre un recurso. En una API, los métodos son la forma de expresar las operaciones CRUD. (Mozilla Contributors, s. f.-d).

- GET: Pide datos; no los modifica. CRUD: Read
- POST: Envía datos para crear un recurso nuevo. CRUD: Create
- PUT: Reemplaza un recurso existente con los datos enviados. CRUD: Update
- DELETE: Elimina un recurso. CRUD: Delete

PUT está diseñado para reemplazar un recurso completo. Al enviar una PUT, se proporciona una representación completa del recurso que se desea actualizar. Si el recurso ya existe, PUT se reemplaza con los nuevos datos. Si no existe, PUT se puede crear un nuevo recurso en la URL especificada.

En cambio, este PATCH se utiliza para realizar actualizaciones parciales de un recurso. Con PATCH, solo se envían los campos que necesitan actualizarse, dejando el resto del recurso sin cambios. Esto resulta especialmente útil para modificaciones menores en las que enviar la representación completa del recurso sería ineficiente.

Códigos de estado. ¿Qué significan 200, 201, 404, 422 y 500? ¿Cuándo los verías al usar FastAPI?

Código	Nombre	Significado	¿Cuándo lo verías en FastAPI?
200	OK	La petición funcionó y se devuelve el resultado.	Es el código por defecto. Lo verías cuando una ruta como GET /personas/2 se ejecuta sin errores.
201	CREATED	Se creó un recurso nuevo.	En un POST /personas que crea un registro. FastAPI no lo pone solo: hay que indicarlo con status_code=201 en el decorador.
404	NOT FOUND	El recurso o la dirección no existe.	Cuando pides un id que no está en los datos (GET /personas/999). También aparece si la ruta no existe.
422	UNPROCESSABLE ENTITY	La petición está bien escrita, pero los datos no son válidos.	FastAPI lo genera automáticamente cuando los datos no cumplen el modelo Pydantic, por ejemplo, edad: "abc" en lugar de un número, o cuando falta un campo obligatorio.
500	INTERNAL SERVER ERROR	Falló algo dentro del servidor.	Cuando el código tiene un error que nadie capturó, como una división entre cero o una falla al conectarse a la base de datos.

Aplicación al módulo. Para el dataset que usas en clase, propone en una tabla 5 endpoints (método + ruta + qué devuelve), por ejemplo GET personas/{id} o POST /personas. No hace falta programarlos.

En HTTP, la respuesta incluye un código de estado numérico de tres dígitos, y los códigos de la familia 200 indican respuestas exitosas (FastAPI, s. f.-a). Los de la familia 400 indican un error por parte del cliente (FastAPI, s. f.-b)

#	Método	Ruta	CRUD	Qué devuelve	Códigos posibles
1	GET	/personas	Read	Lista de todas las personas. Acepta filtros opcionales, por ejemplo /personas?ciudad=Quito&respuesta=Sí	200
2	GET	/personas/{id}	Read	Los datos de una sola persona (JSON con id, nombre, edad, ciudad, respuesta)	200, 404 si el id no existe, 422 si el id no es un número
3	POST	/personas	Create	La persona recién creada, con el id asignado. Recibe un JSON con nombre, edad, ciudad y respuesta	201, 422 si faltan campos o tienen el tipo incorrecto
4	PUT	/personas/{id}	Update	La persona con los datos ya modificados. PUT recibe el registro completo y PATCH solo los campos que cambian	200, 404, 422
5	DELETE	/personas/{id}	Delete	Sin contenido, o un mensaje de confirmación	204, 404

Referencias

FastAPI. (s. f.-a). *Código de estado del response*. <https://fastapi.tiangolo.com/es/tutorial/response-status-code/>

FastAPI. (s. f.-b). *Manejo de errores*. <https://fastapi.tiangolo.com/es/tutorial/handling-errors/>

Monkey Plus. (2026, 27 de mayo). *Mejores pasarelas de pago en Ecuador 2026*. <https://monkeyplusbc.com/contenido/pasarelas-de-pago-ecuador-guia-negocios>

Mozilla Contributors. (s. f.-a). *API*. MDN Web Docs. <https://developer.mozilla.org/es/docs/Glossary/API>

Mozilla Contributors. (s. f.-b). *CRUD*. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Glossary/CRUD>

Mozilla Contributors. (s. f.-c). *JSON*. MDN Web Docs. <https://developer.mozilla.org/es/docs/Glossary/JSON>

Mozilla Contributors. (s. f.-d). *Métodos de petición HTTP*. MDN Web Docs. <https://developer.mozilla.org/es/docs/Web/HTTP/Methods>

Python Software Foundation. (s. f.). *json — Codificador y decodificador JSON*. <https://docs.python.org/es/3.11/library/json.html>